

Document Management System

AI-classified, access-controlled, retention-managed document repository

PRD • Architecture • Workflows • Data Models • API • Roadmap • Cursor Prompts

Current Repository Status

The repo is a **wired scaffold**, not empty. The tech choices are made, and `server.js` + `App.jsx` already import files that do not exist yet. "From scratch" means filling in this skeleton in the right order.

Layer	Exists	Referenced but missing
Backend	<code>server.js</code> , <code>seeds/seedUsers.js</code>	<code>config/db</code> , <code>middleware/errorHandler</code> , <code>models/*</code> , <code>routes/auth documents notifications</code> , <code>utils/hotFolderWatcher</code> , <code>jobs/retentionCron</code> , <code>seeds/seed.js</code> , <code>seeds/seedDocumentRules.js</code>
Frontend	<code>App.jsx</code> , <code>AppShell.jsx</code> , <code>Login.jsx</code> , <code>Register.jsx</code> , <code>utils/api.js</code>	<code>utils/auth</code> , pages: <code>Profile</code> , <code>Notifications</code> , <code>DropFolder</code> , <code>ReviewQueue</code> , <code>MetadataSearch</code> , <code>DocumentDetail</code>

1. What You Are Building

A Document Management System that auto-ingests files, uses AI (OCR + LLM) to classify them, files them into a `Client → Year → Trip → Category` folder tree, tags them with metadata, and enforces role-based access plus retention rules - the 8 core components of a DMS.

Tech stack (already chosen by installed packages)

Concern	Tool	Location
Frontend	React 19 + Vite + React Router 7 + Tailwind 4	frontend/
Backend	Node + Express 5	server/
Database	MongoDB + Mongoose 9	server/
Auth	JWT (jsonwebtoken) + bcryptjs	server/
File upload	multer	server/
Hot-folder watcher	chokidar	server/utils/
AI classification	openai SDK → Google Gemini endpoint	server/services/
OCR / text extract	pdf-parse	server/services/
Retention scheduler	node-cron	server/jobs/
Security	helmet, cors, morgan	server/

2. PRD - Product Requirements Document

2.1 Vision

Replace scattered shared-drive chaos with one centralized, searchable, access-controlled, AI-organized repository where a file dropped in a folder ends up correctly filed, tagged, and retention-managed with zero manual effort.

2.2 Personas & Roles (RBAC)

- **Admin** - full access; manages users, taxonomy rules, retention rules; sees all departments.
- **Manager** - approves classifications in the review queue; sees their department's documents; runs reports.
- **Employee** - uploads and searches documents they have access to.

2.3 Functional Requirements (mapped to the 8 components)

#	Component	Requirements
1	Document Capture	Manual drag-drop upload; hot-folder watcher auto-ingests files dropped on disk; (future: email/scanner/API).
2	AI Classification	Extract text (pdf-parse/OCR) → LLM → return type, folderPath, tags, confidence. Confidence ≥ 85 auto-files; else review queue.
3	Folder & Taxonomy	Hierarchical path Client → Year → Trip → Category → File; folders created on demand; rules configurable.
4	Metadata & Tagging	Stores type, vendor, date, amount, client, department, retention period, relationship type (owned/shared/bonded).
5	Search	Full-text search + faceted filter on any metadata field.
6	Access Control	Role + department based; permission inheritance folder → file; every access logged.
7	Retention & Lifecycle	Per-type retention rule (3yr/7yr/permanent); daily cron warns before expiry, archives or expires; bonded docs locked.
8	Audit & Compliance	Every action logged (who/what/when); one-click audit pack export.

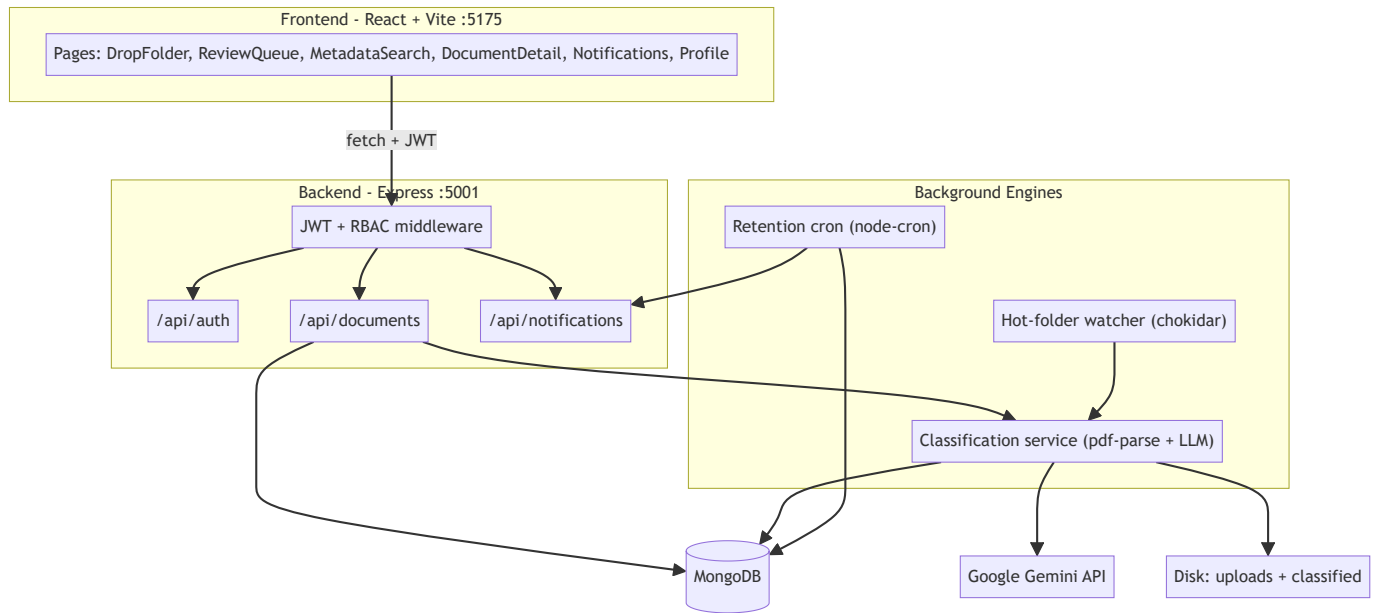
2.4 Non-functional

Sub-second search, JWT auth on all `/api` routes, file validation (type/size), responsive UI, daily retention sweep.

2.5 Out of scope (v1)

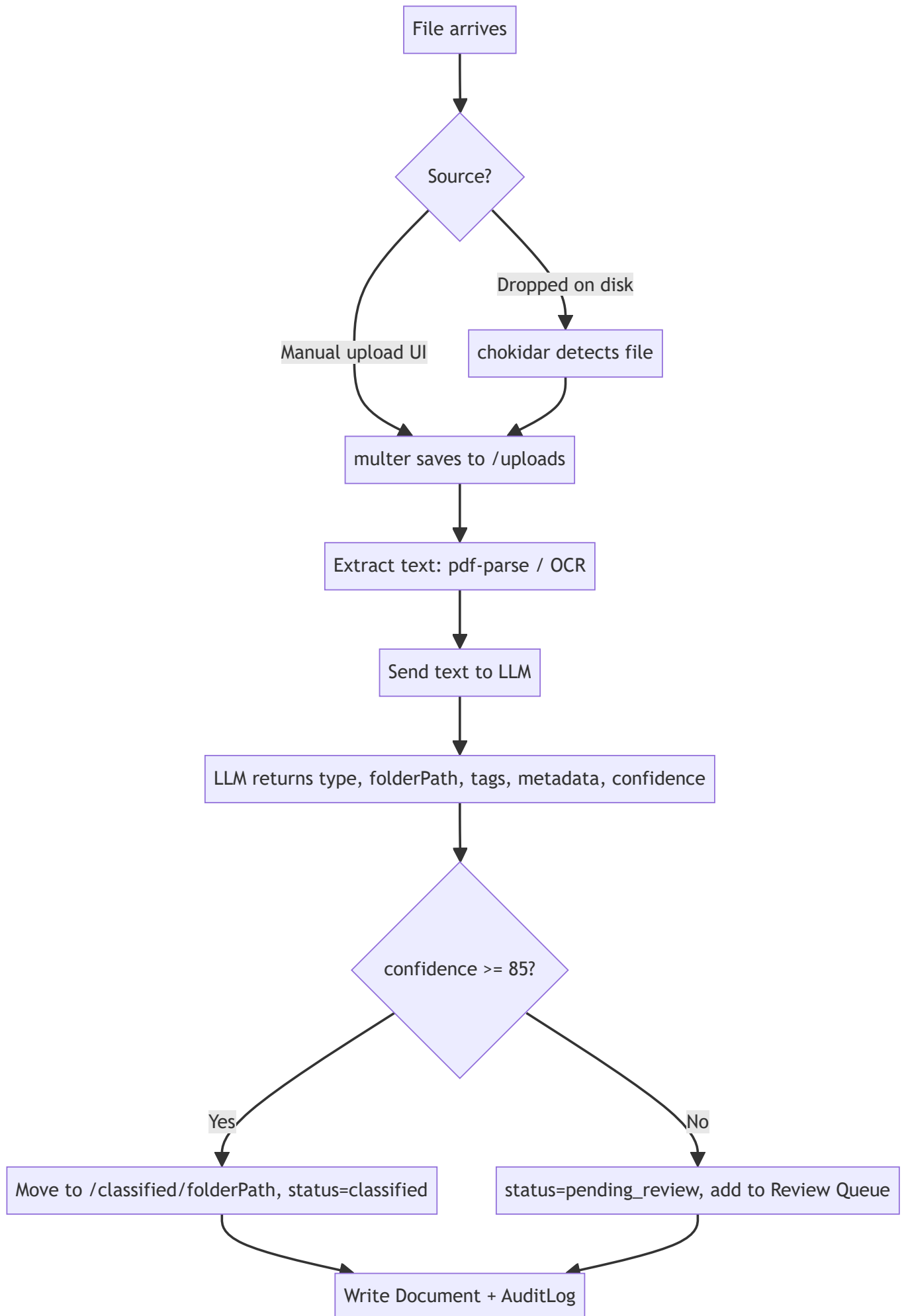
Email/scanner ingestion, eSign approval, Elasticsearch (use Mongo text index first), cold-storage archival.

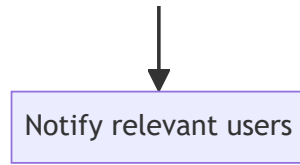
3. System Architecture



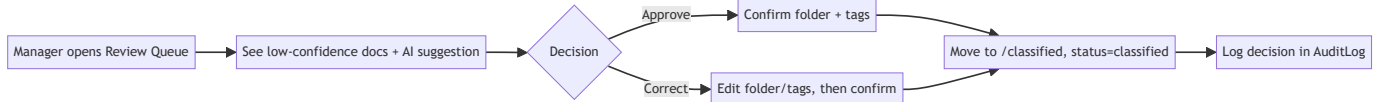
4. Core Workflows

4.1 Upload + AI Classification

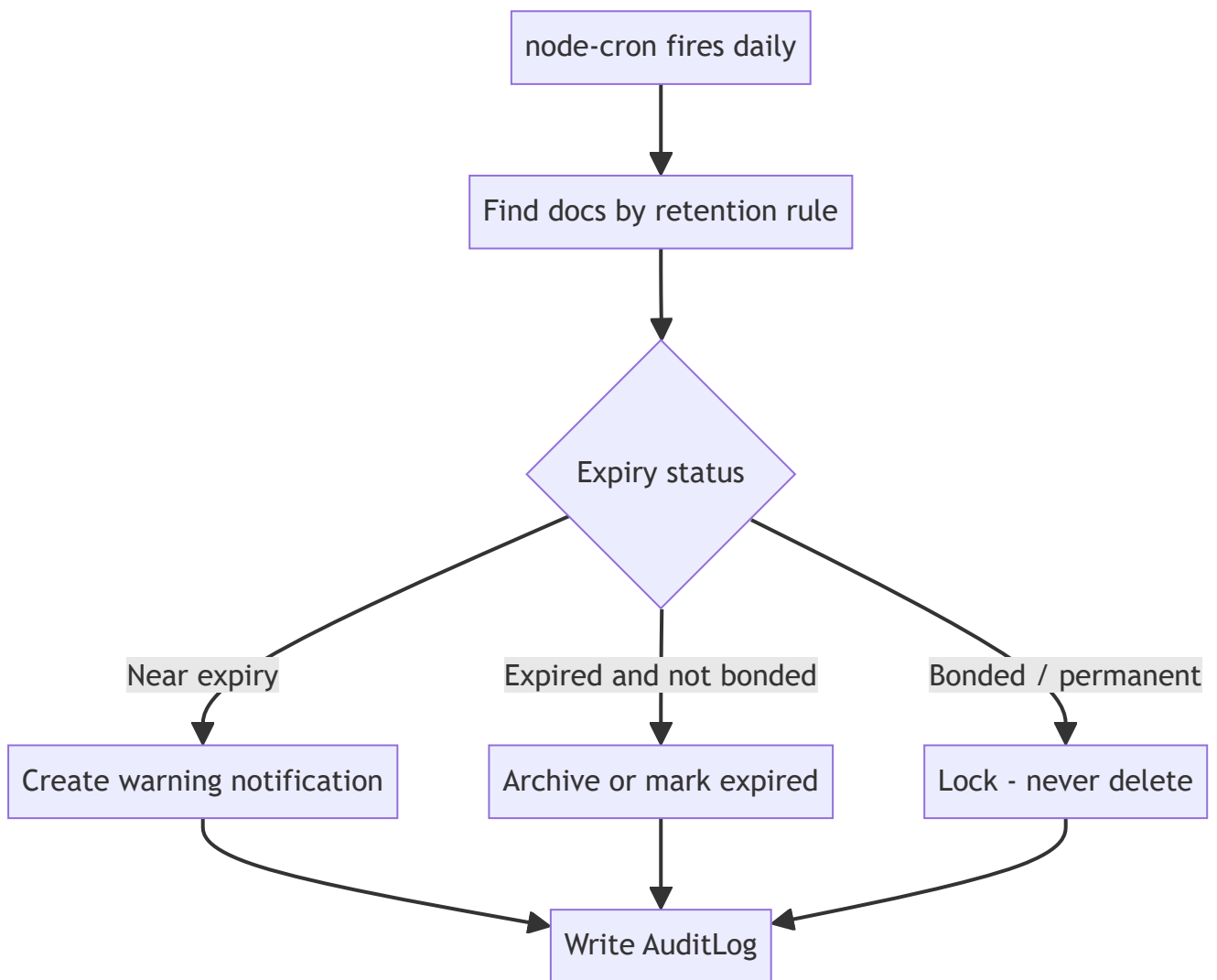




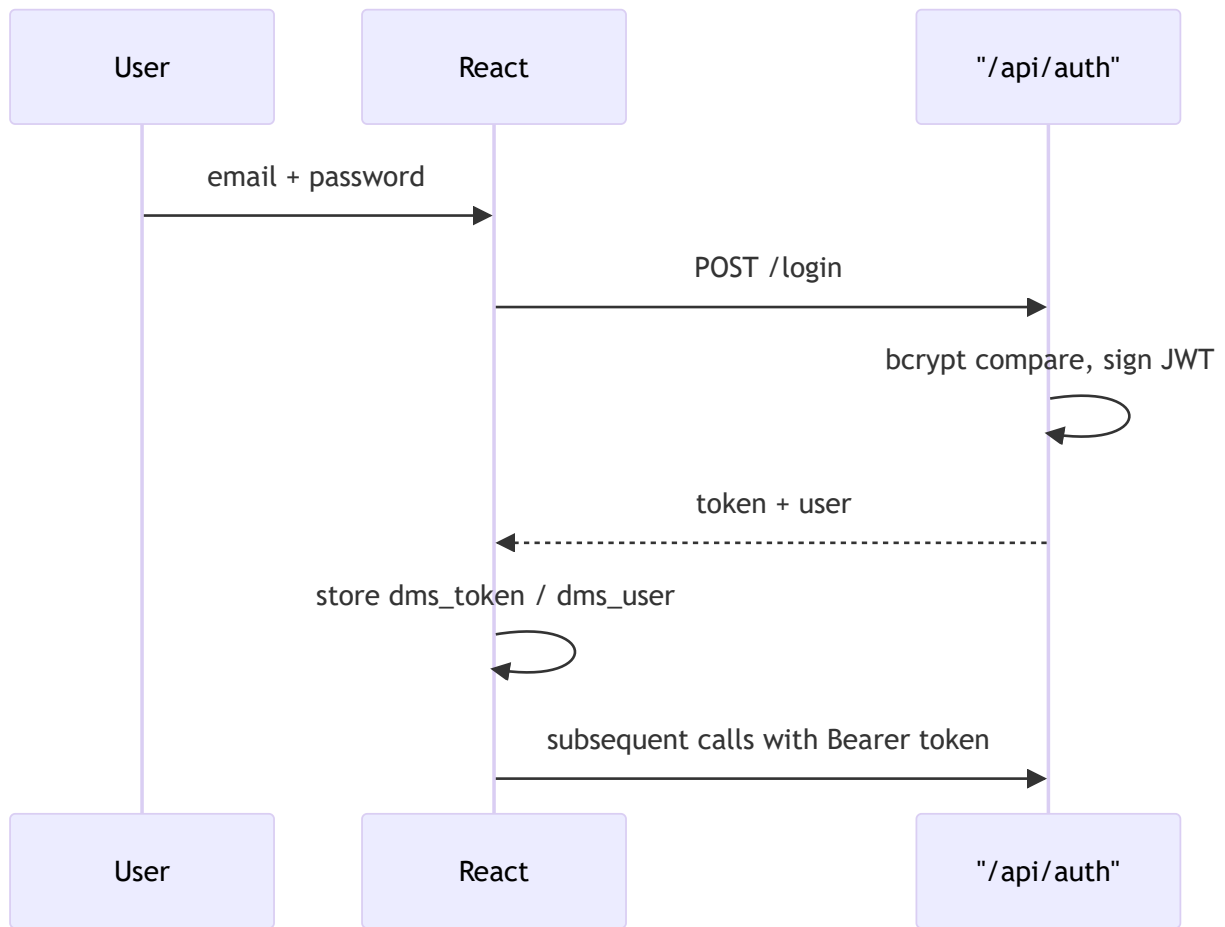
4.2 Review Queue (human-in-the-loop)



4.3 Retention Lifecycle (daily cron)



4.4 Authentication



5. Data Models (Mongoose)

```
// User (partially implied by seedUsers.js)
{ name, email, passwordHash, role: 'Admin'|'Manager'|'Employee',
  department, createdAt }

// Document
{ originalName, storedPath, mimeType, size,
  documentType,           // invoice, contract, HR letter...
  folderPath,             // "AcmeCorp/2026/Trip-12/Invoices"
  client, year, trip, category,
  metadata: { vendor, date, amount, retentionPeriod, relationshipType },
  tags: [String],
  status: 'pending_review'|'classified'|'archived'|'expired',
  confidence: Number,
  uploadedBy: ref(User),
  retention: { policy, expiresAt, locked: Boolean },
  createdAt, updatedAt }

// TaxonomyRule (seedDocumentRules.js feeds this)
{ documentType, folderTemplate, defaultTags, retentionYears }

// AuditLog
{ action, actor: ref(User), documentId: ref(Document),
  details, ip, timestamp }

// Notification
{ user: ref(User), type, message, read: Boolean, createdAt }
```

Add a Mongo **text index** on Document (originalName , tags , metadata.vendor) for full-text search.

6. API Surface

```
POST /api/auth/register      POST /api/auth/login      GET /api/auth/me
GET /api/documents          # list + filter/facets/search (?q=&type=&client=)
POST /api/documents/upload  # multer single file → classify
GET /api/documents/:id
PATCH /api/documents/:id    # edit metadata
GET /api/documents/review/queue # pending_review items
POST /api/documents/:id/review # approve/correct classification
GET /api/notifications      PATCH /api/notifications/:id/read
GET /api/health              # already exists
```

7. Target Folder Structure

```
server/  
  config/db.js  
  middleware/{auth.js, rbac.js, errorHandler.js, upload.js}  
  models/{User.js, Document.js, TaxonomyRule.js, AuditLog.js, Notification.js}  
  routes/{auth.js, documents.js, notifications.js}  
  services/{classifier.js, textExtract.js, taxonomy.js}  
  utils/hotFolderWatcher.js  
  jobs/retentionCron.js  
  seeds/{seed.js, seedUsers.js, seedDocumentRules.js}  
  uploads/      classified/      hot-folder/  
frontend/src/  
  pages/{Login, Register, DropFolder, ReviewQueue, MetadataSearch, DocumentDetail, Notifications, Profile}  
  components/AppShell.jsx  
  utils/{api.js, auth.js}
```

8. Build Roadmap (milestones for Cursor)

Phase	Goal	Deliverable
0. Foundation	DB connect, error handler, User model	config/db.js, middleware/errorHandler.js, models/User.js - server boots
1. Auth	Register/login/me + JWT + bcrypt	routes/auth.js, middleware/auth.js, frontend utils/auth.js - login works
2. Upload + storage	multer upload, Document model	middleware/upload.js, models/Document.js, basic routes/documents.js
3. AI classification	text extract + LLM → folderPath/tags	services/textExtract.js, services/classifier.js
4. Hot-folder + taxonomy	auto-ingest + rules	utils/hotFolderWatcher.js, models/TaxonomyRule.js, seeds
5. Review queue	human-in-the-loop UI + API	ReviewQueue.jsx + review endpoints
6. Search + metadata	text index + faceted filter	MetadataSearch.jsx + list endpoint
7. RBAC + audit	role middleware + AuditLog	middleware/rbac.js, models/AuditLog.js
8. Retention + notifications	cron + alerts	jobs/retentionCron.js, routes/notifications.js

Build in this order - each phase depends on the previous, and it matches how `server.js` / `App.jsx` already reference things.

9. How to Drive Cursor (prompts per phase)

Use one focused prompt per milestone (in Agent mode). Reference existing files by name so Cursor matches your conventions and does not break the wiring.

Phase 0 - Foundation

```
Create server/config/db.js that connects to MONGODB_URI with Mongoose and logs success/failure. Create server/middleware/errorHandler.js matching the { success, error, code } envelope used in frontend/src/utils/api.js. Create server/models/User.js with the fields used in seeds/seedUsers.js (name, email, passwordHash, role, department) and a bcrypt comparePassword method. Don't break the existing server.js requires.
```

Phase 1 - Auth

```
Implement server/routes/auth.js with register, login, and GET /me using JWT and bcrypt. Create frontend/src/utils/auth.js exposing authStore and useUser as imported in App.jsx. Wire Login.jsx and Register.jsx to the api helper.
```

Phase 3 - AI classification

```
Create server/services/textExtract.js (pdf-parse for PDFs, plain read for txt) and server/services/classifier.js that uses the openai SDK pointed at LLM_BASE_URL / LLM_API_KEY / LLM_MODEL from .env. It takes extracted text and returns { documentType, folderPath, client, year, trip, category, tags, metadata, confidence }. If confidence ≥ CLASSIFY_CONFIDENCE_THRESHOLD, auto-file; else mark pending_review.
```

Tip: keep .env set (Gemini key + threshold=85 already there). Build phases 0 through 8 in order.